

TASK 11

FULL APPLICATION CONTAINERIZATION

Dockerized Deep Learning Solution: Model + Flask API + Streamlit UI

Course	MSc Artificial Intelligence
Task	11 - Full Application Containerization
Author	Saikumar Padala
Application	MNIST CNN Digit Recognition
Container Orchestration	Docker Compose

Table of Contents

Section	Page
1. Introduction and Objectives	3
2. System Architecture	4
3. Project Structure and Docker Configuration	5
4. Application Components and Execution	6
5. Testing and Validation	7
6. Optimization and Deployment Readiness	8
7. Results, Observations and Conclusion	9
Appendix A. Source Code	10-16
Appendix B. Screenshot Evidence	17-26
Appendix C. Final Submission Checklist	27

1. Introduction and Objectives

Task 11 extends the Flask deep learning API into a complete containerized application. The solution is split into two cooperating services: a Flask inference backend that loads the trained MNIST CNN model, and a Streamlit frontend that provides the browser-based user interface. Docker Compose builds and runs both services on a shared bridge network.

The important integration boundary is the service-to-service request. Streamlit does not call localhost for the backend when running in its own container; it uses the Docker Compose service name flask-api and port 5000. The Flask API processes the image, performs model inference, and returns JSON containing the predicted digit, confidence, and probability distribution.

Objectives

- Package the model, Flask API and Streamlit UI as Dockerized services.
- Configure reproducible dependencies and isolated runtime environments.
- Build and orchestrate the services using Docker Compose.
- Verify API health, frontend access, prediction and inter-container communication.
- Document architecture, testing evidence, optimization practices and deployment readiness.

Technology Stack

Component	Technology	Role
Frontend	Streamlit	Image upload, prediction display and visualization
Backend	Flask + Gunicorn	REST API and model inference
Model	TensorFlow / Keras CNN	MNIST digit classification
Image Processing	Pillow + NumPy	Grayscale conversion, resize and normalization
Orchestration	Docker Compose	Builds, networks and service lifecycle
Runtime	Python 3.12 slim	Container base image

2. System Architecture

The application uses a browser-facing Streamlit container and a Flask API container on the same Docker Compose network. The trained model is packaged inside the API image because inference is performed by Flask.

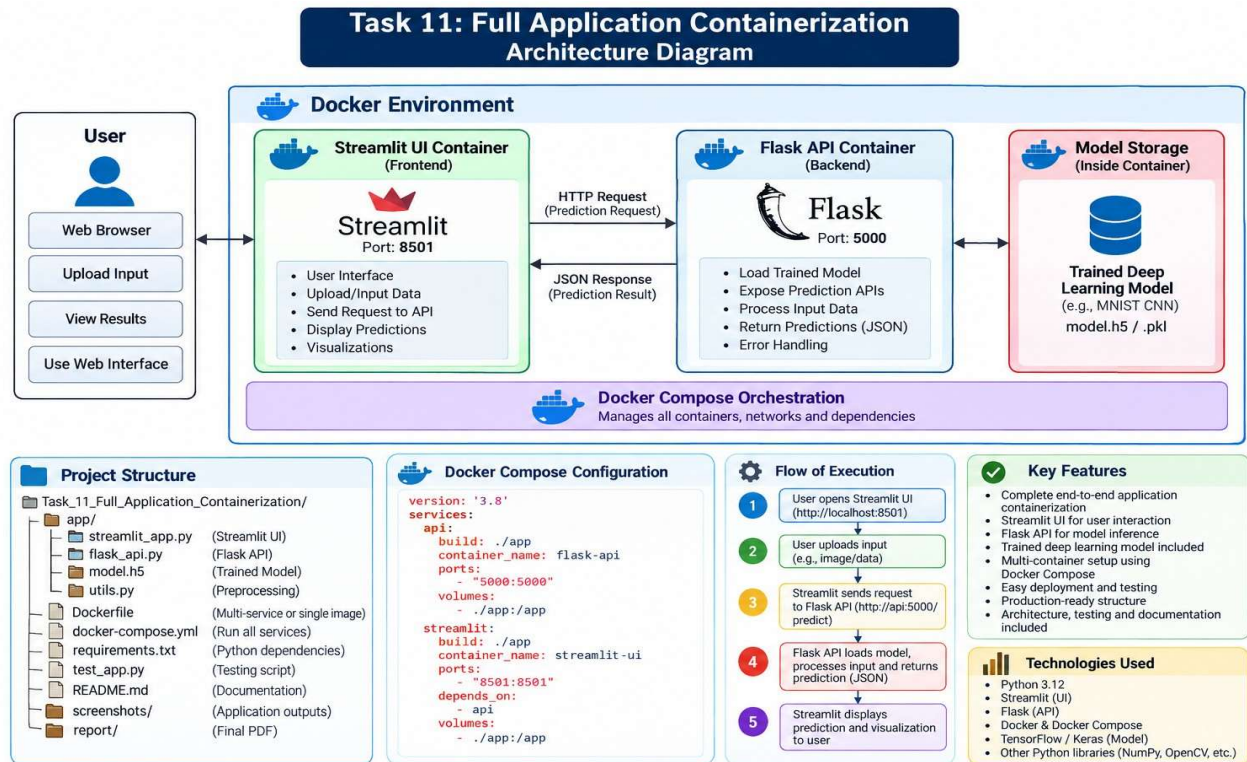


Figure 1. Task 11 full-application containerization architecture.

Execution flow: Browser -> Streamlit UI container -> HTTP POST /predict -> Flask API container -> deep_learning_model.h5 -> JSON response -> Streamlit result display.

3. Project Structure and Docker Configuration

The implementation separates backend and frontend responsibilities. The API build context contains the Flask application, requirements, Dockerfile and the compatible trained model. The UI build context contains the Streamlit application and its Python dependencies.

Project Structure

```
Task_11_Full_Application_Containerization/
|-- api/
|   |-- flask_api.py
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- .dockerignore
|   |-- deep_learning_model.h5
|-- ui/
|   |-- app.py
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- .dockerignore
|-- docker-compose.yml
|-- test_app.py
|-- DOCKER_COMMANDS.txt
|-- screenshots/
`-- report/
```

Docker Compose Configuration

```
services:
  flask-api:
    build:
      context: ./api
      dockerfile: Dockerfile
    image: task11-flask-api:latest
    container_name: task11-flask-api
    ports:
      - "5000:5000"
    networks:
      - task11-network

  streamlit-ui:
    build:
      context: ./ui
      dockerfile: Dockerfile
    image: task11-streamlit-ui:latest
    container_name: task11-streamlit-ui
    environment:
      FLASK_API_URL: http://flask-api:5000
    depends_on:
      - flask-api
    ports:
      - "8501:8501"
    networks:
      - task11-network

networks:
  task11-network:
    driver: bridge
```

4. Application Components and Execution

Flask API

The Flask service loads `deep_learning_model.h5` once at startup, validates uploaded files, converts images to grayscale, fits them to 28 x 28 pixels, normalizes pixel values to the range [0,1], performs CNN inference and returns structured JSON. The API exposes `/`, `/health` and `/predict`.

Streamlit UI

The Streamlit service accepts PNG, JPG and JPEG images. It reads `FLASK_API_URL` from the Compose environment and posts the selected file to `http://flask-api:5000/predict`. The response is rendered as the predicted digit, confidence percentage and probability chart.

Execution Procedure

Step	Command / Action	Purpose
1	Copy <code>deep_learning_model.h5</code> to <code>api/</code>	Package the trained model
2	<code>docker compose build</code>	Build both service images
3	<code>docker compose up -d</code>	Start both containers
4	<code>docker compose ps</code>	Verify service state and health
5	Open <code>http://localhost:8501</code>	Access Streamlit UI
6	Test <code>/health</code> and <code>/predict</code>	Validate API behavior
7	<code>python test_app.py</code>	Automated smoke validation
8	<code>docker stats / logs / network test</code>	Collect deployment evidence

Key configuration used in the live test: Streamlit exposed on port 8501; Flask API exposed on port 5000; the UI used the service name `flask-api` for internal communication.

5. Testing and Validation

The following results are grounded in the screenshots captured during the completed local Docker Compose run.

Test	Observed Result	Status
Docker Compose build	task11-flask-api:latest and task11-streamlit-ui:latest built successfully; model COPY step completed.	PASS
Compose status	Both task11-flask-api and task11-streamlit-ui were Up and reported healthy.	PASS
Flask health	Health response returned success=true, status=healthy and model_loaded=true.	PASS
Streamlit UI	Browser UI loaded on localhost:8501 and accepted 7.png.	PASS
End-to-end prediction	Streamlit displayed Predicted Digit: 7 with 99.99% confidence.	PASS
Flask prediction JSON	POST /predict returned success=true, predicted_digit=7 and confidence_percent=99.99.	PASS
Automated validation	Flask /health -> 200; Flask / -> 200; Streamlit /_stcore/health -> 200.	PASS
Resource usage	Flask approx. 0.03% CPU / 229.0 MiB; Streamlit approx. 0.84% CPU / 131.2 MiB.	PASS
Compose logs	Gunicorn started; model loaded from /app/deep_learning_model.h5; Streamlit server started.	PASS
Inter-container communication	From streamlit-ui, request to http://flask-api:5000/health returned healthy JSON.	PASS

6. Optimization and Deployment Readiness

- Slim Python 3.12 base images reduce unnecessary operating-system content.
- The API uses a multi-stage build so build-time artifacts are not copied into the runtime image.
- pip is run with --no-cache-dir to avoid retaining package caches.
- .dockerignore files reduce build context and exclude report/screenshots and local artifacts.
- Containers run under non-root users (appuser / appuser).
- Gunicorn is used for the Flask runtime instead of the Flask development server.
- Docker HEALTHCHECK instructions provide service health visibility.
- The model is loaded once at API startup rather than for every request.
- Docker Compose uses a dedicated bridge network and service-name DNS.
- FLASK_API_URL is injected through the Compose environment rather than hard-coding container IP addresses.

Measured Image / Runtime Observation

The completed Task 11 run demonstrated approximately 229.0 MiB memory use for the Flask container and 131.2 MiB for the Streamlit container. These measurements are operational observations from docker stats rather than performance benchmarks.

Deployment Readiness

For local validation, the application meets the requested container integration requirements. For production deployment, additional concerns include authentication, TLS termination, structured logging, monitoring, rate limiting, secret management, model versioning and a CI/CD pipeline.

7. Results, Observations and Conclusion

Results

The completed run shows a functional two-service deep learning application. Docker Compose successfully built and started the Flask inference backend and Streamlit frontend. The backend loaded the trained CNN model, the UI accepted 7.png, and the prediction workflow returned digit 7 with an observed confidence of 99.99%. Automated endpoint validation returned HTTP 200 for Flask /health, Flask / and Streamlit /_stcore/health.

Observations

The architecture cleanly separates the user interface from model inference. Service discovery is handled by Docker Compose, so the Streamlit container reaches the backend at flask-api:5000. The final network evidence confirms that this path works. The TensorFlow runtime contributes substantial image and memory footprint, so future optimization could consider a lighter inference runtime or model conversion after compatibility testing.

Conclusion

Task 11 successfully containerizes the complete deep learning solution as a Docker Compose application. The model, Flask API and Streamlit UI are integrated into reproducible services with health checks, non-root execution, service-name communication and documented testing evidence. The result is suitable as a practical demonstration of end-to-end container integration and deployment readiness.

Appendix A. Source Code - Flask API (lines 1-45)

```
from pathlib import Path
from io import BytesIO
import os

import numpy as np
from PIL import Image, ImageOps
from flask import Flask, jsonify, request
from tensorflow.keras.models import load_model

BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = Path(os.getenv("MODEL_PATH", BASE_DIR / "deep_learning_model.h5"))

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 10 * 1024 * 1024

if not MODEL_PATH.exists():
    raise FileNotFoundError(
        f"Model file not found: {MODEL_PATH}. "
        "Place deep_learning_model.h5 beside flask_api.py or set MODEL_PATH."
    )

model = load_model(MODEL_PATH, compile=False)
print(f"CNN model loaded successfully from: {MODEL_PATH}")

def preprocess_image(file_bytes: bytes) -> np.ndarray:
    """Prepare an uploaded image for the trained MNIST CNN."""
    image = Image.open(BytesIO(file_bytes)).convert("L")
    image = ImageOps.fit(image, (28, 28), method=Image.Resampling.LANCZOS)
    pixels = np.asarray(image, dtype=np.float32) / 255.0

    input_shape = model.input_shape
    if len(input_shape) == 4:
        return pixels.reshape(1, 28, 28, 1)
    if len(input_shape) == 3:
        return pixels.reshape(1, 28, 28)
    raise ValueError(f"Unsupported model input shape: {input_shape}")

@app.get("/")
def index():
    return jsonify({
        "success": True,
        "message": "MNIST CNN Flask API is running inside Task 11 Docker Compose.",
        "endpoints": {"health": "/health", "predict": "/predict"},
    })
```

Appendix A. Source Code - Flask API (lines 46-90)

```
   )), 200

@app.get("/health")
def health():
    return jsonify({
        "success": True,
        "status": "healthy",
        "model_loaded": model is not None,
    }), 200

@app.post("/predict")
def predict():
    if "file" not in request.files:
        return jsonify({
            "success": False,
            "error": "Missing image. Send the uploaded image in form field 'file'.",
        }), 400

    uploaded = request.files["file"]
    if not uploaded.filename:
        return jsonify({"success": False, "error": "No file selected."}), 400

    try:
        image_bytes = uploaded.read()
        if not image_bytes:
            return jsonify({"success": False, "error": "Uploaded file is empty."}), 400

        model_input = preprocess_image(image_bytes)
        probabilities = np.asarray(model.predict(model_input, verbose=0))[0]
        predicted_digit = int(np.argmax(probabilities))
        confidence = float(probabilities[predicted_digit])

        return jsonify({
            "success": True,
            "predicted_digit": predicted_digit,
            "confidence": confidence,
            "confidence_percent": round(confidence * 100, 2),
            "probabilities": [float(p) for p in probabilities],
        }), 200

    except Exception:
        app.logger.exception("Prediction failed")
        return jsonify({
```

Appendix A. Source Code - Flask API (lines 91-105)

```
        "success": False,
        "error": "Could not process this image. Please upload a valid PNG, JPG, or JPEG.",
    }, 400

@app.errorhandler(413)
def file_too_large(_error):
    return jsonify({
        "success": False,
        "error": "File is too large (maximum 10 MB).",
    }), 413

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.getenv("PORT", "5000")), debug=False)
```

Appendix A. Source Code - Streamlit UI

```
import os
import requests
import streamlit as st
import pandas as pd
from PIL import Image

API_BASE_URL = os.getenv("FLASK_API_URL", "http://127.0.0.1:5000").rstrip("/")

st.set_page_config(page_title="MNIST Digit Recognition - Task 11", page_icon="[digit]", layout="centered")
st.title("MNIST Digit Recognition")
st.subheader("Task 11 - Full Dockerized Application")
st.write(
    "Upload a handwritten digit image. Streamlit runs in its own Docker container "
    "and sends the image to the Flask API container for CNN prediction."
)
st.divider()

uploaded_file = st.file_uploader("Upload a handwritten digit image", type=["png", "jpg", "jpeg"])

if uploaded_file is not None:
    st.image(uploaded_file, caption="Uploaded Image", width=280)
    if st.button("Predict Digit", type="primary"):
        try:
            with st.spinner("Sending image to the Flask API..."):
                response = requests.post(
                    f"{API_BASE_URL}/predict",
                    files={
                        "file": (
                            uploaded_file.name,
                            uploaded_file.getvalue(),
                            uploaded_file.type or "application/octet-stream",
                        )
                    },
                    timeout=30,
                )

            if response.status_code != 200:
                try:
                    message = response.json().get("error", response.text)
                except ValueError:
                    message = response.text
                st.error(f"Prediction request failed ({response.status_code}): {message}")
            else:
                result = response.json()
                if not result.get("success", False):
                    st.error(result.get("error", "The API did not return a successful result. "))
                else:
                    digit = int(result["predicted_digit"])
                    confidence = float(result["confidence"])
                    probabilities = result["probabilities"]
                    st.success(f"Predicted Digit: {digit}")
                    st.metric("Prediction Confidence", f"{confidence * 100:.2f}%")
                    chart_data = pd.DataFrame({
                        "Digit": list(range(10)),
                        "Probability": probabilities,
                    }).set_index("Digit")
                    st.subheader("Prediction Probabilities")
                    st.bar_chart(chart_data)
        except requests.exceptions.ConnectionError:
            st.error("Cannot connect to Flask API. Verify that both Docker services are running.")
        except requests.exceptions.Timeout:
            st.error("The prediction request timed out. Please try again.")
        except (ValueError, KeyError, TypeError) as exc:
            st.error(f"The API response was not in the expected format: {exc}")
    else:
        st.info("Choose a PNG, JPG, or JPEG image to begin.")

st.divider()
st.info(f"Backend API: {API_BASE_URL}")
```

Appendix A. Source Code - Dockerfiles

API Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    PATH="/opt/venv/bin:$PATH"

WORKDIR /app
RUN apt-get update && \
    apt-get install -y --no-install-recommends libgomp1 && \
    rm -rf /var/lib/apt/lists/*

COPY --from=builder /opt/venv /opt/venv
COPY flask_api.py .
COPY deep_learning_model.h5 .

RUN useradd --create-home --uid 10001 appuser && \
    chown -R appuser:appuser /app
USER appuser

EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --start-period=45s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:5000/health', timeout=3)"

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "--threads", "2", "--timeout", "120", "flask_api:app"]
```

UI Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .

RUN useradd --create-home --uid 10002 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8501
HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8501/_stcore/health', timeout=3)"

CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501", "--server.headless=true"]
```

Appendix A. Source Code - docker-compose.yml

```
services:
  flask-api:
    build:
      context: ./api
      dockerfile: Dockerfile
    image: task11-flask-api:latest
    container_name: task11-flask-api
    ports:
      - "5000:5000"
    networks:
      - task11-network

  streamlit-ui:
    build:
      context: ./ui
      dockerfile: Dockerfile
    image: task11-streamlit-ui:latest
    container_name: task11-streamlit-ui
    environment:
      FLASK_API_URL: http://flask-api:5000
    depends_on:
      - flask-api
    ports:
      - "8501:8501"
    networks:
      - task11-network

networks:
  task11-network:
    driver: bridge
```

Appendix A. Source Code - Validation and Requirements

test_app.py

```
import json
import urllib.request

API = "http://127.0.0.1:5000"
UI = "http://127.0.0.1:8501"

def get_json(url):
    with urllib.request.urlopen(url, timeout=5) as response:
        return response.status, json.loads(response.read().decode("utf-8"))

if __name__ == "__main__":
    status, payload = get_json(API + "/health")
    print("Flask /health ->", status)
    print(json.dumps(payload, indent=2))

    status, payload = get_json(API + "/")
    print("Flask / ->", status)
    print(json.dumps(payload, indent=2))

    with urllib.request.urlopen(UI + "/_stcore/health", timeout=5) as response:
        print("Streamlit /_stcore/health ->", response.status)
```

API requirements.txt

```
Flask==3.1.2
gunicorn==26.2.0
numpy>=1.26,<2.3
Pillow>=11,<13
tensorflow-cpu==2.21.0
```

UI requirements.txt

```
streamlit>=1.49,<2
requests>=2.32,<3
pandas>=2.2,<3
Pillow>=11,<13
```

Appendix B. Screenshot Evidence 1

```
C:\Windows\System32\cmd.exe
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker compose build

[+] Building 1807.1s (31/31) FINISHED
=> [internal] load local image definitions
=> => reading from stdin 1.41kB
=> [flask-api internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.17kB
=> [streamlit-ui internal] load build definition from Dockerfile
=> => transferring dockerfile: 695B
=> [streamlit-ui] resolve image config for docker-image://docker.io/docker/dockerfile:1
=> [auth] docker/dockerfile:pull token for registry-1.docker.io
=> CACHED [flask-api] docker-image://docker.io/docker/dockerfile:1sha256:4edf897a3ffa55b89f986fc8cc78afdb3f1834cc9c7083565e611a8a7d5fe99e
=> => resolve docker.io/docker/dockerfile:1sha256:4edf897a3ffa55b89f986fc8cc78afdb3f1834cc9c7083565e611a8a7d5fe99e
=> [flask-api internal] load metadata for docker.io/library/python:3.12-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [streamlit-ui internal] load .dockerignore
=> => transferring context: 89B
=> [flask-api internal] load .dockerignore
=> => transferring context: 89B
=> [flask-api internal] load build context
=> => transferring context: 1.51MB
=> [flask-api builder 1/5] FROM docker.io/library/python:3.12-slimsha256:85cda9777409a9c3ffdd694a4c74b679f6765a8b8857f9c7ed9226b6806b6d6f
=> => resolve docker.io/library/python:3.12-slimsha256:85cda9777409a9c3ffdd694a4c74b679f6765a8b8857f9c7ed9226b6806b6d6f
=> [streamlit-ui internal] load build context
=> => transferring context: 68B
=> CACHED [flask-api runtime 2/7] WORKDIR /app
=> [streamlit-ui 3/6] COPY requirements.txt
=> [streamlit-ui 4/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [flask-api builder 2/5] WORKDIR /build
=> CACHED [flask-api builder 3/5] RUN python -m venv /opt/venv
=> CACHED [flask-api builder 4/5] COPY requirements.txt
=> [flask-api builder 5/5] RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt
=> [streamlit-ui 5/6] COPY app.py
=> [streamlit-ui 6/6] RUN useradd --create-home --uid 10002 appuser && chown -R appuser:appuser /app
=> [streamlit-ui] exporting to image
=> => exporting layers
=> => exporting manifest sha256:91e5abbf397a61228b08ecfbb784158899f606214c497273c69685469c7cc92b
=> => exporting config sha256:a309dee27b8c61cab54595ae38892bc318816cald8bb8f1f1470f4ade35fc0b2
=> => exporting attestation manifest sha256:2cc4a3f83982749acccecc322ce8e1db6f9b3d5d9966a33b9f9e8956edaf
=> => exporting manifest list sha256:861db6cfd9db81433908a4f487721b852939bb447c8cbff646215677defda9c1
=> => naming to docker.io/library/task11-streamlit-ui:latest
=> => unpacking to docker.io/library/task11-streamlit-ui:latest
=> [streamlit-ui] resolving provenance for metadata file
=> CACHED [flask-api runtime 3/7] RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 && rm -rf /var/lib/apt/lists/*
=> [flask-api runtime 4/7] COPY --from=builder /opt/venv /opt/venv
=> [flask-api runtime 5/7] COPY flask_api.py
=> [flask-api runtime 6/7] COPY deep_learning_model.hs
=> [flask-api runtime 7/7] RUN useradd --create-home --uid 10001 appuser && chown -R appuser:appuser /app
=> [flask-api] exporting to image
=> => exporting layers
=> => exporting manifest sha256:a9a2eccc933204b878168559cf23a39f468095e1c5b862ef433b9ec0d1aad44
=> => exporting config sha256:12853f3d8981172465a9885a3c8984a2bb892d4f688eeea9149679783fc16aa59e
=> => exporting attestation manifest sha256:3e2f68218666f6c9b42b3a54d4af331e8d282c1e13266d385c9296ab51e
=> => exporting manifest list sha256:13288b9b088a777e7e61db52ce27c831fe8dd49b4e98aec207688bdc7a35cc0
=> => naming to docker.io/library/task11-flask-api:latest
=> => unpacking to docker.io/library/task11-flask-api:latest
=> [flask-api] resolving provenance for metadata file
[+] 2/2d 2/2
=> image task11-flask-api:latest Built
=> image task11-streamlit-ui:latest Built

C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>
```

Figure 2. Docker Compose build output - both application images built successfully.

Appendix B. Screenshot Evidence 2

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker compose up -d
[*] up 2/2
  ✓ Container task11-flask-api Started 0.4s
  ✓ Container task11-streamlit-ui Started 1.9s

C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
task11-flask-api    task11-flask-api:latest  "gunicorn --bind 0.0.0.0"  flask-api  2 minutes ago  Up 51 seconds (healthy)  5000/tcp
task11-streamlit-ui task11-streamlit-ui:latest "streamlit run app.py"    streamlit-ui  2 minutes ago  Up 50 seconds (healthy)  0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp

C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>
```

Figure 3. Docker Compose service status - Flask API and Streamlit UI running and healthy.

Appendix B. Screenshot Evidence 3

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>curl.exe http://127.0.0.1:5000/health  
{"model_loaded":true,"status":"healthy","success":true}
```

Figure 4. Flask health endpoint - success=true, status=healthy and model_loaded=true.

Appendix B. Screenshot Evidence 4

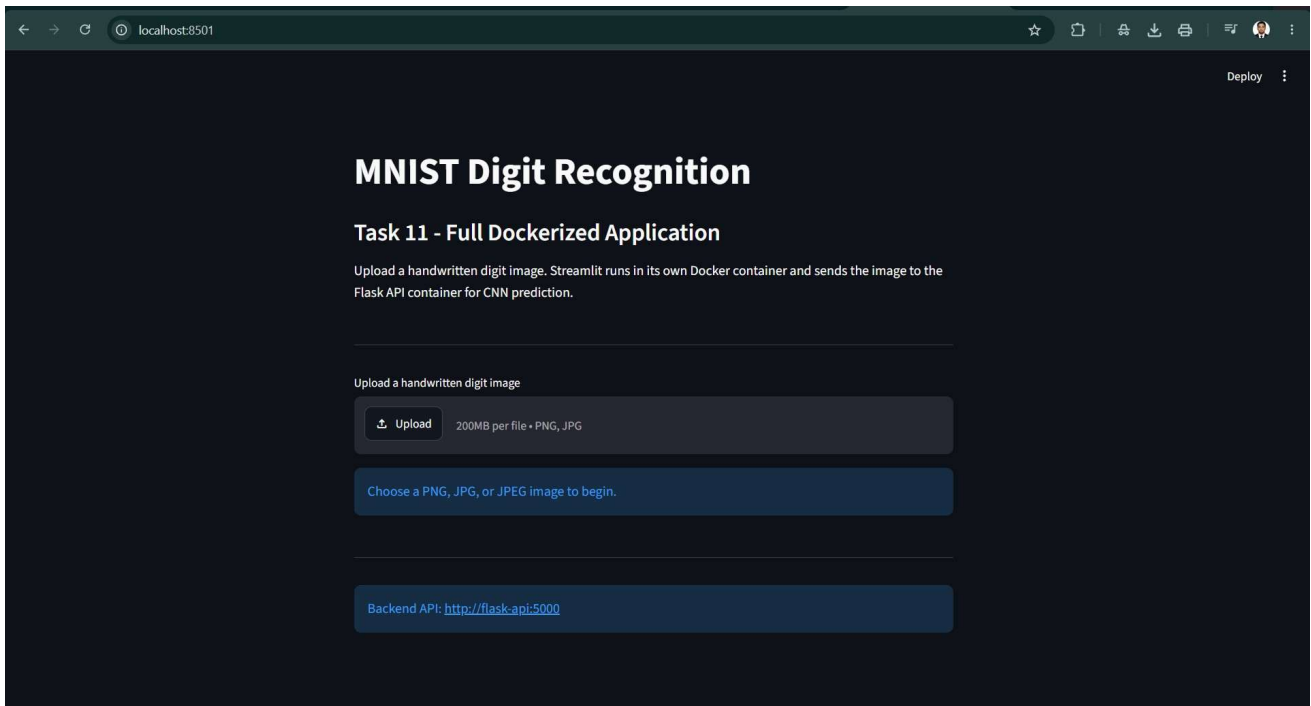


Figure 5. Streamlit UI home page running on localhost:8501.

Appendix B. Screenshot Evidence 5

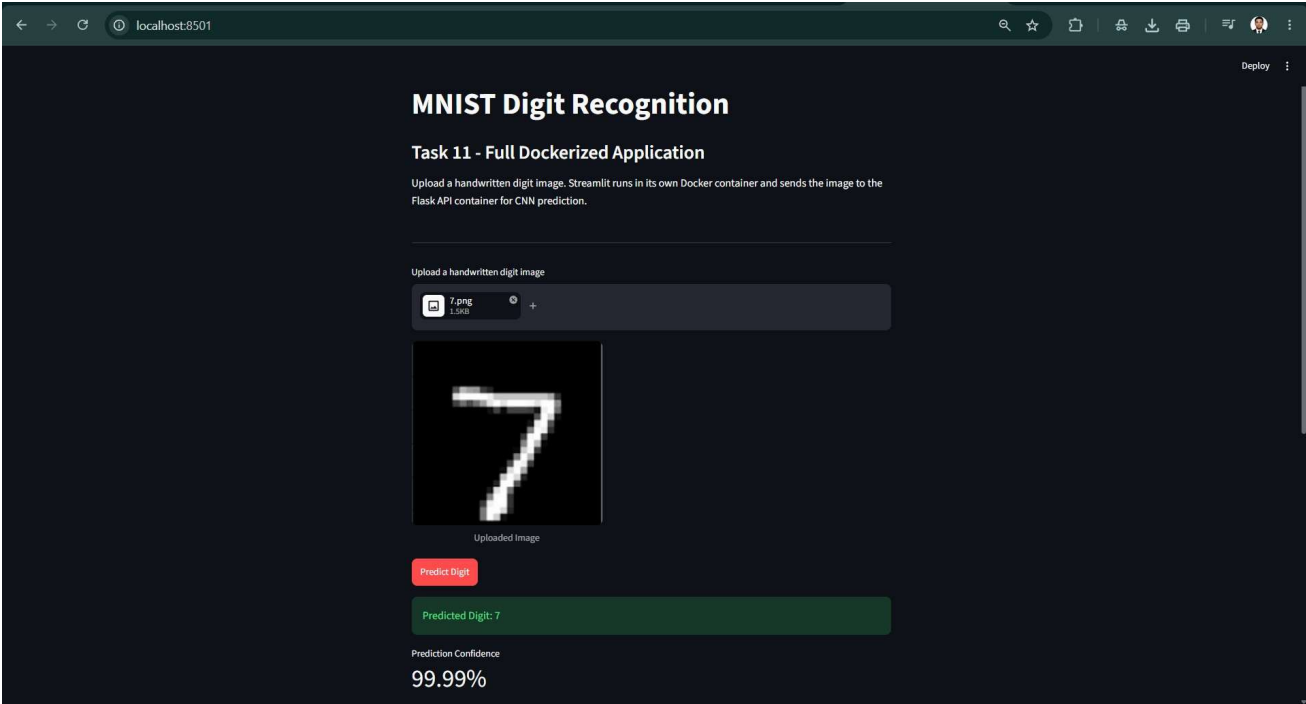


Figure 6. End-to-end Streamlit prediction - digit 7 with 99.99% confidence.

Appendix B. Screenshot Evidence 6

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>curl.exe -X POST "http://127.0.0.1:5000/predict" -F "file=@7.png"
{"confidence":0.9999245405197144,"confidence_percent":99.99,"predicted_digit":7,"probabilities":[8.011453189737949e-08,9.70641735875688e-07,1.2208653288325877e-06,5.929720282438211e-05,1.2992479314632988e-09,2.5290702865277126e-07,1.9525459265916367e-13,0.9999245405197144,9.634394615432029e-08,1.3667156963492744e-05],"success":true}
```

Figure 7. Flask POST /predict JSON response - successful digit 7 prediction.

Appendix B. Screenshot Evidence 7

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>python test_app.py
Flask /health -> 200
{
  "model_loaded": true,
  "status": "healthy",
  "success": true
}
Flask / -> 200
{
  "endpoints": {
    "health": "/health",
    "predict": "/predict"
  },
  "message": "MNIST CNN Flask API is running inside Task 11 Docker Compose.",
  "success": true
}
Streamlit /_stcore/health -> 200
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>
```

Figure 8. Automated application validation - Flask and Streamlit health checks returned HTTP 200.

Appendix B. Screenshot Evidence 8

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker stats --no-stream task11-flask-api task11-streamlit-ui
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
2e9a2fa502dd	task11-flask-api	0.03%	229.3MiB / 2.783GiB	8.05%	7.91kB / 3.89kB	10.7MB / 4.1kB	30
ce09f7fd0105	task11-streamlit-ui	0.04%	131.2MiB / 2.783GiB	4.60%	138kB / 5.39MB	103MB / 4.1kB	20

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>
```

Figure 9. Docker resource usage for both containers.

Appendix B. Screenshot Evidence 9

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker compose logs --tail=100 flask-api
task11-flask-api | [2026-10-07 16:11:25 +0000] [1] [INFO] Starting gunicorn 26.2.0
task11-flask-api | [2026-10-07 16:11:25 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
task11-flask-api | [2026-10-07 16:11:25 +0000] [1] [INFO] Using worker: gthread
task11-flask-api | [2026-10-07 16:11:25 +0000] [7] [INFO] Booting worker with pid: 7
task11-flask-api | [2026-10-07 16:11:26 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicornctl
task11-flask-api | WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
task11-flask-api | I9800 00:00:1791389492.958306 7 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations
task11-flask-api | To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
task11-flask-api | CNN model loaded successfully from: /app/deep_learning_model.h5

C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker compose logs --tail=100 streamlit-ui
task11-streamlit-ui | Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
task11-streamlit-ui | 2026-10-07 16:06:12.568 Uvicorn server started on 0.0.0.0:8501
task11-streamlit-ui | Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
task11-streamlit-ui | 2026-10-07 16:06:34.196 Uvicorn server started on 0.0.0.0:8501
task11-streamlit-ui | You can now view your Streamlit app in your browser.
task11-streamlit-ui | Local URL: http://localhost:8501
task11-streamlit-ui | Network URL: http://172.18.0.2:8501
task11-streamlit-ui | External URL: http://103.172.226.63:8501
task11-streamlit-ui | Detected WSL. Using poll-based file watching for better compatibility. To force watchdog, set server.fileWatcherType = "watchdog".
task11-streamlit-ui | Stopping...
task11-streamlit-ui | Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
task11-streamlit-ui | 2026-10-07 16:11:28.679 Uvicorn server started on 0.0.0.0:8501
task11-streamlit-ui | You can now view your Streamlit app in your browser.
task11-streamlit-ui | Local URL: http://localhost:8501
task11-streamlit-ui | Network URL: http://172.18.0.3:8501
task11-streamlit-ui | External URL: http://103.172.226.63:8501
task11-streamlit-ui | Detected WSL. Using poll-based file watching for better compatibility. To force watchdog, set server.fileWatcherType = "watchdog".

C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>
```

Figure 10. Docker Compose logs - Gunicorn, model loading and Streamlit startup evidence.

Appendix B. Screenshot Evidence 10

```
C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>docker compose exec streamlit-ui python -c "import urllib.request; print(urllib.request.urlopen('http://flask-api:5000/health').read().decode())"
{"model_loaded":true,"status":"healthy","success":true}

C:\Users\padal\Downloads\Task_11_Full_Application_Containerization\Task_11_Full_Application_Containerization>
```

Figure 11. Definitive container-to-container network test - Streamlit container reached flask-api:5000/health successfully.

Appendix C. Final Submission Checklist

- Source code files present: Flask API, Streamlit UI, Dockerfiles, Compose and test script.
- Trained model referenced as `deep_learning_model.h5` in `api/`.
- Docker Compose build and service startup validated.
- Health, prediction, UI and automated tests validated.
- Resource usage and logs captured.
- Container-to-container communication validated using the `flask-api` service name.
- Architecture diagram included.
- Screenshot evidence included in chronological order.
- Final PDF suitable for LMS submission or inclusion in the public GitHub repository.

GitHub link for LMS:

<https://github.com/Saipadala2003/task11-full-dockerized-mnist>